

WooCommerce Field Guide

WooCommerce, Subscriptions and Stripe order, renewal and payment fixes.

138 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 138 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/woocommerce-fixes

Guides: allanninal.dev/woocommerce

All 14 repos: github.com/allanninal

The fixes

1 Action Scheduler stuck in-progress

One action never finished, and now it is sitting on in-progress forever. Renewals stop firing, retry emails stop going out, and the queue behind it gets longer every hour. This is what makes an action freeze mid run, why it can block the queue, and a small script that finds every stuck action, checks Stripe for the truth, and tells you exactly which ones are safe to reset.

[audit_stuck_actions.py](#)

<https://www.allanninal.dev/woocommerce/action-scheduler-stuck-in-progress/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/action-scheduler-stuck-in-progress>

2 WooCommerce Action Scheduler tables that balloon in size

The site admin gets slower every month. Backups take longer to run. Then someone opens the database and finds `wp_actionscheduler_actions` and `wp_actionscheduler_logs` sitting at millions of rows, dwarfing every other table in the store. Action Scheduler is supposed to clean up after itself, but that cleanup depends on cron running every day without fail, and it only takes one bad week for the tables to start piling up for good. Here is why it happens and a small script that reports the size and clears out old completed actions safely.

[audit_action_scheduler.py](#)

<https://www.allanninal.dev/woocommerce/action-scheduler-tables-balloon/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/action-scheduler-tables-balloon>

3 Active sub with a past next payment date

The subscription looks fine in the admin list. Status: Active. But open it up and the next payment date is last week, or last month, and no renewal order ever appeared. The customer is not being charged, you are not getting paid, and nothing in the UI is telling you why. Here is why the schedule falls behind while the status stays green, and a small job that finds every case and moves the date forward safely.

[fix_past_next_payment.py](#)

<https://www.allanninal.dev/woocommerce/active-sub-with-a-past-next-payment-date/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/active-sub-with-a-past-next-payment-date>

4 Active-subscriber counts wrong in reports

The dashboard says one number. A count of real subscriptions says another. Nobody changed anything on purpose, yet the "Active subscribers" widget and the actual list of paying customers have quietly gone out of sync. Here is why the cached total drifts and a small script that recounts from real subscriptions and repairs the report safely.

[recount_active_subscribers.py](#)

<https://www.allanninal.dev/woocommerce/active-subscriber-counts-wrong-in-reports/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/active-subscriber-counts-wrong-in-reports>

5 Add an off session mandate to old WooCommerce subscriptions

A subscription that has renewed cleanly for years suddenly starts failing. The card has not expired, the customer has not disputed anything, and Stripe rejects the charge with `requires_action` instead of taking the money. The subscription was saved before Strong Customer Authentication became mandatory, so it never picked up an off session mandate, the piece of paperwork Stripe now insists on before letting a merchant bill a card while the customer is not there. Here is why it happens and a small job that finds every old subscription missing a mandate and attaches one safely.

[attach_off_session_mandate.py](#)

<https://www.allanninal.dev/woocommerce/add-off-session-mandate-to-old-subs/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/add-off-session-mandate-to-old-subs>

6 WooCommerce orders where the amount does not match the order

The order shows Processing or Completed, so it looks fine at a glance. But the number Stripe actually captured and the number on the WooCommerce order total quietly disagree. Nobody notices until a refund is short, a customer disputes a charge, or the books do not add up at the end of the month. Here is why the two numbers drift apart and a small script that compares them, order by order, and tells you exactly which ones to look at.

[check_amount_mismatch.py](#)

<https://www.allanninal.dev/woocommerce/amount-does-not-match-the-order/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/amount-does-not-match-the-order>

7 **Attach the payment method**

A customer has a saved card in WooCommerce, the checkout even shows it, but the next renewal or off session charge fails with a Stripe error about the PaymentMethod. Stripe created the PaymentMethod when the card was added, but it was never attached to the Customer object. Here is why that gap opens up and a small script that finds every affected customer and attaches the PaymentMethod before it costs you a missed renewal.

[attach_payment_method.py](#)

<https://www.allanninal.dev/woocommerce/attach-the-payment-method/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/attach-the-payment-method>

8 **WooCommerce attribute and term counts drift from the real products**

A shopper filters by "Color: Blue" and the widget says 42 products, but only 31 actually show up. Nobody touched the theme. Nobody touched the filter plugin. The number stored on the attribute term simply stopped matching the catalog. Here is why that count drifts and a small script that recounts every term and repairs the ones that are wrong.

[recount_terms.py](#)

<https://www.allanninal.dev/woocommerce/attribute-and-term-counts-drift/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/attribute-and-term-counts-drift>

9 **WooCommerce authorized Stripe charges that are never captured**

Your store is set to authorize the card now and capture the money later, maybe so you can check stock or screen for fraud first. The order goes on hold with the money reserved, waiting for someone to press capture. Then it gets forgotten. About seven days later Stripe releases the hold, the reserved money vanishes, and you shipped or owe an order you were never paid for. Here is why it happens and a small job that captures the valid ones before the window closes.

[capture_authorized.py](#)

<https://www.allanninal.dev/woocommerce/authorized-charges-never-captured/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/authorized-charges-never-captured>

10 **Autoloaded options bloat**

Every page on your store, the storefront, the cart, even a plain REST API call, loads a bundle of "autoloaded" settings before it does anything else. Too many autoloaded options, or a few very large ones, and every one of those pages gets slower. This is a quiet problem because nothing errors out. It just gets a little heavier every week. Here is why it happens and a small script that finds the big rows and trims the ones that are safe to trim.

[find_stale_autoload.py](#)

<https://www.allanninal.dev/woocommerce/autoloaded-options-bloat/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/autoloaded-options-bloat>

11 **Backfill order ID metadata**

A reconciliation script asks Stripe for succeeded PaymentIntents and matches each one to a WooCommerce order using metadata.order_id. That works fine for anything paid recently. Then you run it against older orders and half of them come back as orphans, not because the payment failed, but because the PaymentIntent was created before your store ever wrote an order id into its metadata. Here is why that gap exists and a small script that backfills it safely.

[backfill_order_id_metadata.py](#)

<https://www.allanninal.dev/woocommerce/backfill-order-id-metadata/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/backfill-order-id-metadata>

12 **WooCommerce Subscriptions still try to bill after WooPayments is turned off**

You disabled WooPayments, maybe to switch to a different processor or to pause the store, and assumed billing stopped with it. It did not. WooCommerce Subscriptions keeps its own renewal schedule, and every subscription still points at the payment method it was created with. The scheduled action fires anyway, tries to charge through a gateway that no longer exists, and fails quietly. Here is why that happens and a small script that finds every subscription still wired to the disabled gateway and moves it to manual renewal so it stops trying.

[stop_billing_on_disabled_gateway.py](#)

<https://www.allanninal.dev/woocommerce/billing-runs-after-woopayments-off/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/billing-runs-after-woopayments-off>

13 **Broken featured images on WooCommerce products**

A shopper opens a product page, or an order confirmation email lands in an inbox, and instead of the product photo there is a small broken image icon. The product is real, the price is right, the listing still sells, but the picture is gone. This is one of the quieter WooCommerce problems, because nothing crashes and no error shows up in a log. Here is why it happens and a small script that finds every product pointing at a missing file and clears it safely.

[repair_broken_images.py](#)

<https://www.allanninal.dev/woocommerce/broken-featured-images/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/broken-featured-images>

14 **Bulk cancel WooCommerce subscriptions without leaving Stripe still billing**

You select forty subscriptions in the WooCommerce admin, choose Cancel from the bulk action menu, and the list refreshes with every row showing Cancelled. Job done, you think. Then a customer writes in three weeks later asking why Stripe just charged their card for a subscription they cancelled. The WooCommerce side moved. The Stripe side did not. Here is why a bulk cancel can leave the two systems disagreeing, and a small script that cancels a batch on both sides and tells you exactly what it did.

[bulk_cancel_sync.py](#)

<https://www.allanninal.dev/woocommerce/bulk-cancel-and-keep-in-sync/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/bulk-cancel-and-keep-in-sync>

15 **Bulk pause WooCommerce subscriptions and Stripe billing together**

A support queue asks you to pause a batch of subscriptions, maybe fifty customers going on a payment holiday, maybe a whole plan getting retired for a quarter. You bulk edit them to On hold in WooCommerce and move on. A week later Stripe has quietly billed every one of them anyway, because pausing a subscription in WooCommerce and pausing it in Stripe are two different actions, and only one of them actually stops the charge.

[bulk_pause.py](#)

<https://www.allanninal.dev/woocommerce/bulk-pause-on-both-systems/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/bulk-pause-on-both-systems>

16 Bulk subscription export runs out of memory

You click "Export all subscriptions" on a store with tens of thousands of them, and a few minutes later the process is just gone. No file, no error a normal person can read, sometimes a log line about an allowed memory size being exhausted. The export was never broken by bad data, it was broken by trying to hold every subscription in memory at the same time. Here is why that happens and a small script that exports in safe, paged batches instead.

[export_subscriptions.py](#)

<https://www.allanninal.dev/woocommerce/bulk-subscription-export-runs-out-of-memory/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/bulk-subscription-export-runs-out-of-memory>

17 Bulk swap a reissued card range across WooCommerce Subscriptions

A bank or card network reissues a whole range of numbers after a breach, and every subscription still charging one of the old cards is about to fail its next renewal. That can be dozens or thousands of subscriptions at once, all breaking for the same reason on the same week. Here is why it happens and a small script that finds every affected subscription and moves it onto a clean card in a safe way.

[swap_reissued_card.py](#)

<https://www.allanninal.dev/woocommerce/bulk-swap-a-reissued-card-range/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/bulk-swap-a-reissued-card-range>

18 Clean up abandoned Stripe PaymentIntents cluttering WooCommerce

Every time a shopper opens checkout, Stripe can create a PaymentIntent. The ones who leave without paying leave those intents behind, showing as Incomplete in the Stripe dashboard, each with a matching pending order in WooCommerce. Over time they pile up, inflate your open-order count, and make your conversion look worse than it is. This guide cancels the ones that are truly abandoned, and leaves real payments alone.

[cancel_abandoned.py](#)

<https://www.allanninal.dev/woocommerce/cancel-abandoned-payment-intents/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/cancel-abandoned-payment-intents>

19 WooCommerce Subscriptions will not let the customer change the card a second time

A customer updates the card on their subscription. It works, the confirmation email goes out, everyone moves on. A month later they try to update it again, maybe because the old card expired, and this time nothing happens. The form submits, the page reloads, and the subscription is still charging the same dead card. Here is why the second change fails and a small script that finds every subscription in this state and clears the block.

[clear_stale_card.py](#)

<https://www.allanninal.dev/woocommerce/cannot-change-the-card-twice/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/cannot-change-the-card-twice>

20 Cannot reactivate a pending-cancel WooCommerce Subscription

The customer changed their mind. Support clicks Reactivate. WooCommerce Subscriptions says no. The subscription stays on pending-cancel, still ticking down toward the end date it was scheduled to cancel on. This is a small, well-known trap in the subscription status machine, and it is fixed with two writes in the right order, not one forced status change.

[reactivate_pending_cancel.py](#)

<https://www.allanninal.dev/woocommerce/cannot-reactivate-a-pending-cancel-sub/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/cannot-reactivate-a-pending-cancel-sub>

21 **Changing the card flips the WooCommerce subscription to Pending**

A customer updated the card on their subscription because the bank asked for a fresh 3D Secure check. They completed the verification, Stripe shows the new card saved and confirmed, and the subscription still says Pending. No renewal has run since, and the customer has quietly stopped being billed while thinking everything is fine. This guide finds those subscriptions and sets them back to active once the card change is proven to have worked.

[resume_pending_after_card_change.py](#)

<https://www.allanninal.dev/woocommerce/card-change-flips-sub-to-pending/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/card-change-flips-sub-to-pending>

22 **Card check read as a failed payment**

A zero amount Stripe card check went through the same failure path as a real renewal charge. The subscription got dunned, the customer got a "your payment failed" email, and support had to explain that nothing was ever actually charged. This guide covers why a \$0 check gets mistaken for a failed payment, and a small script that finds subscriptions punished this way and puts them back where they belong.

[card_check_auditor.py](#)

<https://www.allanninal.dev/woocommerce/card-check-read-as-a-failed-payment/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/card-check-read-as-a-failed-payment>

23 **Card not saved for future renewals**

The first order paid without a hitch. The customer's card went through, the order shows paid, everyone moved on. Months later the renewal runs and fails, because there was never a reusable card behind that first payment in the first place. Here is why that gap opens up and a small script that finds every subscription sitting in it before the next renewal date arrives.

[find_unsaved_renewal_cards.py](#)

<https://www.allanninal.dev/woocommerce/card-not-saved-for-future-renewals/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/card-not-saved-for-future-renewals>

24 **Checkout-draft orders pile up**

Open your orders table and filter by status. If your store uses the block based checkout, there is a good chance you will find hundreds or thousands of orders sitting in checkout-draft, most of them days or months old, none of them ever paid. Nobody deleted them because nothing in WooCommerce core ever does. Here is why they build up and a small script that clears out the stale ones without touching a checkout that is still in progress.

[purge_checkout_drafts.py](#)

<https://www.allanninal.dev/woocommerce/checkout-draft-orders-pile-up/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/checkout-draft-orders-pile-up>

25 **The WooCommerce Clear transients tool leaves residue behind**

You ran Status, Tools, Clear transients because wp_options was getting heavy. The count went down, the page said it worked, and for most stores that is the end of the story. But a few rows can survive the sweep with no partner left to match them, and one of those leftovers is a cached Stripe payment status that stops updating. The order looks fine in the list. Stripe disagrees. Here is why the tool misses a few rows and a small script that finds every order the residue is hiding.

[repair_transient_residue.py](#)

<https://www.allanninal.dev/woocommerce/clear-transients-tool-leaves-residue/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/clear-transients-tool-leaves-residue>

26 Completed actions never purged

Open the Scheduled Actions screen and filter by Complete. On a store that has run for a year or two, you will often find hundreds of thousands, sometimes millions, of actions sitting there finished, doing nothing, taking up space. Action Scheduler is supposed to clean these up on its own. On a lot of stores, it quietly falls behind and never catches up. Here is why that happens and a small script that safely purges the old finished ones once it confirms, through Stripe, that nothing important is still riding on them.

[purge_completed_meta.py](#)

<https://www.allanninal.dev/woocommerce/completed-actions-never-purged/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/completed-actions-never-purged>

27 Coupon expiry uses UTC not site time

A shop owner sets a coupon to expire on a certain date, expecting it to work through the end of that day in their own timezone. Instead it stops working hours early, or worse, on what looks like the wrong day entirely. Nothing is broken in the usual sense. WooCommerce is comparing a coupon's expiry against UTC time, not the store's local time, and the two clocks do not agree. Here is why it happens and a small script that finds every coupon expiring on the wrong day and fixes it.

[fix_coupon_expiry_timezone.py](#)

<https://www.allanninal.dev/woocommerce/coupon-expiry-uses-utc-not-site-time/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/coupon-expiry-uses-utc-not-site-time>

28 Coupon usage count undercounts

A coupon is set to a strict usage limit, but somehow it gets used more times than that limit allows. Or the opposite happens, a coupon that has clearly been used a dozen times still shows a usage count of nine. The number WooCommerce stores for a coupon's usage_count has quietly fallen behind the real number of orders that used it. Here is why that number drifts and a small job that recounts it from the orders themselves and repairs it.

[recount_coupon_usage.py](#)

<https://www.allanninal.dev/woocommerce/coupon-usage-count-undercounts/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/coupon-usage-count-undercounts>

29 WooCommerce coupon usage counted twice via the REST API

A shopper uses a coupon once, but the coupon's usage_count climbs by two. Do that enough times and a coupon with a usage_limit of one hundred looks used up at fifty real redemptions, so it locks out shoppers who never even tried it. This is a REST API order handling quirk, not a coupon problem, and it has a small, testable fix.

[coupon_usage_dedupe.py](#)

<https://www.allanninal.dev/woocommerce/coupon-usage-counted-twice-via-rest/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/coupon-usage-counted-twice-via-rest>

30 **Currency not enabled on Stripe**

A shopper checks out in a currency your store now offers, but Stripe was never approved to accept it. The charge is refused, the order sits unpaid, and the error in the logs just says something like `currency_not_enabled`, which tells nobody what to actually do. Here is why it happens and a small script that finds every order at risk before it fails, and every order that already has.

[detect_currency_not_enabled.py](#)

<https://www.allanninal.dev/woocommerce/currency-not-enabled-on-stripe/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/currency-not-enabled-on-stripe>

31 **Customer lifetime value drifts**

A regular buyer emails to ask why the store's records show them spending far less than they know they have. You open the customer in WooCommerce and the lifetime value looks stale, too low, or too high, and it does not match the orders you can see with your own eyes. Lifetime value is a cached number, not a live one, and caches drift. Here is why it happens and a small script that recomputes it correctly from real paid orders.

[recompute_clv.py](#)

<https://www.allanninal.dev/woocommerce/customer-lifetime-value-drifts/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/customer-lifetime-value-drifts>

32 **The WooCommerce customer lookup table is out of sync with real orders**

A customer calls in about a loyalty discount they should have earned. You check their account and it says three orders and a lifetime total that is way too low. You pull up their real order history and count nine orders, most of them paid. The customer is not wrong. The lookup table WooCommerce uses to answer that question fast is out of date, and it will not fix itself. Here is why that table drifts and a small script that rebuilds only the rows that are wrong.

[rebuild_customer_lookup.py](#)

<https://www.allanninal.dev/woocommerce/customer-lookup-out-of-sync/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/customer-lookup-out-of-sync>

33 **A declined card leaves the WooCommerce order stuck on Pending**

A buyer tries to pay, the card is declined, and they give up or try elsewhere. The order should move to Failed, but it stays on Pending. Over time the orders list fills with dead checkouts, stock stays reserved for payments that will never come, and your reports count carts that never converted. This guide finds the declined orders and moves them to Failed, cleanly and safely.

[fail_declined.py](#)

<https://www.allanninal.dev/woocommerce/declined-card-order-stuck-pending/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/declined-card-order-stuck-pending>

34 **Deleting a variation does not resync the parent**

Someone removes a discontinued size or color, and the variable product itself does not catch up. The price range on the shop page keeps quoting the deleted variation's price. The "In stock" badge stays green even though the only variations left are out of stock. Here is why WooCommerce leaves the parent stale and a small script that finds every affected product and resyncs it safely.

[resync_variable_parent.py](#)

<https://www.allanninal.dev/woocommerce/deleting-a-variation-does-not-resync-the-parent/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/deleting-a-variation-does-not-resync-the-parent>

35 **Detect test vs live Stripe key mixups on a WooCommerce store**

Checkout looks fine. The form submits. Then every single charge comes back declined, and the Stripe dashboard shows nothing at all, because you are looking at the wrong dashboard. A test key sitting where a live key should be, or the reverse on a staging site, will reject every real card while looking like a normal integration problem. Here is why the mixup happens and a small script that finds it with certainty, using the store's own order data.

[detect_key_mixup.py](#)

<https://www.allanninal.dev/woocommerce/detect-test-vs-live-key-mixups/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/detect-test-vs-live-key-mixups>

36 **Disputes not recorded on the order**

The bank files a chargeback, Stripe pulls the money out of your balance right away, and the case clock starts ticking. But open the WooCommerce order and it looks exactly like it did the day it was paid. No note, no warning, no deadline. The shop manager finds out from the bank statement, not from the store. Here is why the dispute never reaches the order and a small script that writes it there safely.

[record_disputes.py](#)

<https://www.allanninal.dev/woocommerce/disputes-not-recorded-on-the-order/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/disputes-not-recorded-on-the-order>

37 **WooCommerce reduced stock twice for one order**

A shopper bought three units of a product, and stock dropped by six. Nobody placed a second order, no refund happened, and the order itself still shows quantity three. Somewhere in the background, the same order ran through the stock reduction step twice. Here is why that happens and a small script that finds every order this hit and adds the extra stock back.

[repair_double_stock.py](#)

<https://www.allanninal.dev/woocommerce/double-stock-reduction/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/double-stock-reduction>

38 **Dunning stops before its attempts**

A renewal failed, the subscription went on-hold, and the retry schedule was supposed to try the card again in a day, then a few days after that. Instead nothing happens. No more attempts run, no email goes out, and the subscription just sits there like it is still mid retry. This is what happens when the dunning schedule quietly dies before it uses every attempt it was configured for, and here is a small job that finds those subscriptions and resumes them.

[resume_dunning.py](#)

<https://www.allanninal.dev/woocommerce/dunning-stops-before-its-attempts/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/dunning-stops-before-its-attempts>

39 **Stripe charged the customer twice for one WooCommerce order**

Now and then a customer gets billed twice for a single order. Stripe shows two successful charges a minute apart, both for the same order, while WooCommerce only knows about one. It comes from a race between the browser redirect and the webhook, and it is a fast track to chargebacks. Here is how to find every double charge and refund the extra one in a safe, repeatable way.

[refund_duplicates.py](#)

<https://www.allanninal.dev/woocommerce/duplicate-charge-redirect-webhook-race/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/duplicate-charge-redirect-webhook-race>

40 **Duplicate customer accounts at checkout**

A shopper emails support confused: they have two accounts, and only one of them shows the order they just placed. Nothing looks broken from the outside, the order went through fine, but WooCommerce quietly created a second customer account for the same email during checkout. Here is why a checkout race causes this and a small script that finds every duplicate pair and reports a safe plan to merge them.

[find_duplicate_accounts.py](#)

<https://www.allanninal.dev/woocommerce/duplicate-customer-accounts-at-checkout/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/duplicate-customer-accounts-at-checkout>

41 **Duplicate customers for one email**

A shopper writes in confused. They saved a card last month, and today checkout asks for it again like they were never here. Nothing was lost. Stripe just has two, three, sometimes five customer records for the same email, and the one WooCommerce is looking at right now is not the one holding the card. Here is why that happens and a small script that finds every duplicate and merges them onto one survivor, safely.

[merge_duplicate_customers.py](#)

<https://www.allanninal.dev/woocommerce/duplicate-customers-for-one-email/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/duplicate-customers-for-one-email>

42 **Duplicate or missing SKUs in WooCommerce**

Two products in the catalog quietly share the same SKU, or a product has no SKU at all. Nobody notices until a stock feed matches the wrong item, a report double counts sales, or a warehouse picks the wrong box. WooCommerce never stops this from happening on its own. Here is why it creeps in and a small script that finds every conflict and tells you exactly which ones are safe to fix and which ones need a human to look first.

[sku_audit.py](#)

<https://www.allanninal.dev/woocommerce/duplicate-or-missing-skus/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/duplicate-or-missing-skus>

43 **Duplicate renewal orders in one cycle**

A subscription is due to renew, and instead of one renewal order, it gets two. Same subscription, same billing period, two order numbers. One of them usually charged the card, the other sat there unpaid, confusing the customer and cluttering the order list. Here is why WooCommerce Subscriptions occasionally fires the renewal twice, and a small script that finds every extra order and cancels it, without ever touching one that was actually paid.

[cancel_duplicate_renewals.py](#)

<https://www.allanninal.dev/woocommerce/duplicate-renewal-orders-in-one-cycle/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/duplicate-renewal-orders-in-one-cycle>

44 **Duplicate saved cards on one WooCommerce customer's Stripe profile**

A customer opens their account page to update a payment method and finds the same Visa ending in 4242 listed three times. Nothing is broken from Stripe's point of view. Each one is a separate, valid `PaymentMethod`. But the customer is confused, the renewal picker is cluttered, and support has to guess which copy the next subscription payment will actually use. Here is why the same card keeps getting saved more than once and a small script that finds every duplicate and keeps just one.

[dedupe_saved_cards.py](#)

<https://www.allanninal.dev/woocommerce/duplicate-saved-cards/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/duplicate-saved-cards>

45 **The same Stripe webhook event ran twice and left double notes on the order**

Support flags an order with two identical payment notes, or a customer says they got two "your order shipped" emails, or stock dropped by four when only two were bought. Nothing is actually broken with the payment. The same Stripe event simply got applied to the order more than once. Here is why that happens and a small event ledger that keeps every webhook handler safely idempotent.

[dedupe_webhook_events.py](#)

<https://www.allanninal.dev/woocommerce/duplicate-webhook-events/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/duplicate-webhook-events>

46 **Early renewal shifts the billing cadence**

A customer clicked "Renew now" to pay a subscription early, or a store manager pushed a manual renewal through the REST API. The payment went through fine. But the subscription's next payment date never moved, so it is still anchored to the old cadence. A few days later the customer is charged again, and now every renewal after that is off by the same amount. Here is why the schedule drifts and a small script that realigns it to the date that was actually paid.

[realign_next_payment.py](#)

<https://www.allanninal.dev/woocommerce/early-renewal-shifts-the-billing-cadence/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/early-renewal-shifts-the-billing-cadence>

47 **Expired sale prices never revert**

The sale ended last week. The banner is gone, the email campaign is over, but the product still rings up at the sale price. Nobody changed anything. WooCommerce was supposed to clear it automatically, and most of the time it does, until the one scheduled task that clears it quietly stops firing. Here is why that happens and a small script that finds every product still stuck on an expired sale and reverts it in a safe way.

[revert_expired_sales.py](#)

<https://www.allanninal.dev/woocommerce/expired-sale-prices-never-revert/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/expired-sale-prices-never-revert>

48 **Expired transients bloat wp_options**

Every checkout writes a small, short lived value into the options table so WooCommerce and its Stripe gateway can keep track of things in progress, like a lock that stops one payment from being processed twice. Most of the time that value expires and is quietly forgotten. It is never deleted. On a store that has been running for a year, that forgetting adds up to tens of thousands of dead rows sitting in wp_options, most of them set to load on every single page. Here is why WordPress never cleans them up on its own, and a small script that finds the ones that are safe to clear.

[clear_stale_checkout_locks.py](#)

<https://www.allanninal.dev/woocommerce/expired-transients-bloat-wp-options/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/expired-transients-bloat-wp-options>

49 **Failed order reduces stock, never restored**

A customer's card gets declined, or they abandon checkout, and the order lands on Failed or Cancelled. WooCommerce is supposed to give that stock back automatically. Sometimes it does not, and the count stays down forever. Nobody notices until the product shows "out of stock" while a box of it is still sitting on the shelf. Here is why the restore step gets skipped and a small script that finds every order still holding stock it should have released, and gives it back.

[restore_failed_stock.py](#)

<https://www.allanninal.dev/woocommerce/failed-order-reduces-stock-never-restored/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/failed-order-reduces-stock-never-restored>

50 **Failed orders inflate and lock coupons**

A customer types in a coupon code, the checkout fails, and they try again a minute later, but WooCommerce now says the coupon has already reached its usage limit. Nobody actually used it. The card was declined, or the customer abandoned the Stripe payment page, but WooCommerce had already counted the coupon as spent the moment the order was created. Here is why that count never gets released and a small script that finds every failed order still holding a coupon slot and gives it back.

[release_failed_coupons.py](#)

<https://www.allanninal.dev/woocommerce/failed-orders-inflate-and-lock-coupons/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/failed-orders-inflate-and-lock-coupons>

51 **Fee and net missing on renewals**

A subscription renewal charges the card just fine, WooCommerce Subscriptions marks the order paid, and everything looks normal in the order list. But open the order and the Stripe fee and net fields are blank, while the original signup order right next to it has both filled in. Somewhere an update quietly broke the code that saves those two numbers, only for renewals. Here is why that split happens and a small script that backfills the missing fee and net from Stripe so your books are accurate again.

[backfill_renewal_fees.py](#)

<https://www.allanninal.dev/woocommerce/fee-and-net-missing-on-renewals/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/fee-and-net-missing-on-renewals>

52 Free trials forced to manual renewal

A customer signs up for a free trial, gets a warm welcome email, and everyone forgets about it until the trial ends. That is when support finds out the subscription never actually renewed. Not because the card was declined, but because WooCommerce never had a card to charge in the first place, so it quietly dropped the subscription to manual renewal. Here is why that happens and a small script that finds every subscription stuck this way and puts the ones that can be fixed back on automatic billing.

[restore_trial_billing.py](#)

<https://www.allanninal.dev/woocommerce/free-trials-forced-to-manual-renewal/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/free-trials-forced-to-manual-renewal>

53 Guest orders not linked to accounts

A shopper checks out as a guest using the exact email address they already have an account under, and the order never shows up in "My account". Support gets a "where is my order history" ticket, loyalty points never accrue, and lifetime value reports quietly undercount that customer. Here is why WooCommerce leaves the order unlinked and a small script that relinks it by email, safely.

[link_guest_orders.py](#)

<https://www.allanninal.dev/woocommerce/guest-orders-not-linked-to-accounts/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/guest-orders-not-linked-to-accounts>

54 HPOS schedule divergence

A subscription's next payment date should be one fact. On a store with High Performance Order Storage turned on, it quietly becomes two facts: the value in the HPOS order tables, and an older copy in postmeta kept around for backward compatibility. When something writes to only one of them, the two disagree, and whichever system trusts the stale copy renews, reports, or emails on the wrong day. Here is why it happens and a small script that finds every subscription where the two have drifted apart and brings them back to one source of truth.

[reconcile_schedule_dates.py](#)

<https://www.allanninal.dev/woocommerce/hpos-schedule-divergence/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/hpos-schedule-divergence>

55 iDEAL and Bancontact for renewals

The first order looked completely normal. The customer picked iDEAL or Bancontact at checkout, approved the payment in their banking app, and the order came through paid. Nobody thought about it again until the first renewal date arrived and quietly failed with no card to charge. iDEAL and Bancontact are one off methods, so a subscription that starts on either one is stuck the moment WooCommerce Subscriptions tries to bill it automatically. Here is why that happens and a small script that finds every subscription sitting in this trap and asks the customer for a reusable card before the renewal date, not after.

[check_one_off_methods.py](#)

<https://www.allanninal.dev/woocommerce/ideal-and-bancontact-for-renewals/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/ideal-and-bancontact-for-renewals>

56 **Idempotency gap on PaymentIntents**

A retry went out without an idempotency key, and Stripe had no way to know it was the same payment as the first one. It made a second charge. The customer paid twice, the order shows one payment, and nobody notices until the customer emails you about a strange amount on their card statement. Here is why the gap opens and a small script that finds every likely duplicate so you can review and refund it.

[find_duplicate_intents.py](#)

<https://www.allanninal.dev/woocommerce/idempotency-gap-on-paymentintents/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/idempotency-gap-on-paymentintents>

57 **Import cards from another processor into Stripe, safely**

You switched away from an old processor and Stripe is now the gateway. The problem is every saved card lives on the old processor as a token that means nothing to Stripe. Customers cannot check out, subscriptions cannot renew, and their cards look "missing" even though nothing was ever lost. Here is how a real card migration actually works and a small script that finishes the handoff in WooCommerce without ever touching a raw card number.

[import_migrated_cards.py](#)

<https://www.allanninal.dev/woocommerce/import-cards-from-another-processor/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/import-cards-from-another-processor>

58 **Invisible auto-draft orders**

Open your checkout page and close the tab without paying. Somewhere in your database, an order just got created anyway. Nobody sees it. It never shows in the Orders list, no email goes out, no stock is touched, but the row is there, and it stays there forever unless something removes it. On a busy store this becomes thousands of rows of pure clutter. Here is why it happens and a small script that finds the old ones and cleans them up safely.

[purge_auto_drafts.py](#)

<https://www.allanninal.dev/woocommerce/invisible-auto-draft-orders/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/invisible-auto-draft-orders>

59 **WooCommerce paid orders reverted to failed by a late Stripe event**

An order was paid. The buyer got charged, the order moved to Processing, everyone was happy. Then hours later it flips back to Failed on its own, and stock is released while the money is still with you. The cause is a Stripe failure event for an earlier attempt that arrived late and out of order. Here is why that happens and a small job that finds these orders and restores them in a safe way.

[restore_paid.py](#)

<https://www.allanninal.dev/woocommerce/late-failure-reverts-paid-order/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/late-failure-reverts-paid-order>

60 **Legacy order rows survive after HPOS cleanup**

You turned on High-Performance Order Storage, waited for the sync to finish, and switched off compatibility mode. Months later you notice the old order posts are still sitting in `wp_posts`, unused and untouched, as if the cleanup never ran. This happens more than WooCommerce's own documentation lets on. Here is why it happens and a small script that finds every leftover row and reports it, safely, for cleanup.

[find_legacy_order_rows.py](#)

<https://www.allanninal.dev/woocommerce/legacy-order-rows-survive-after-hpos-cleanup/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/legacy-order-rows-survive-after-hpos-cleanup>

61 **Legacy to new checkout cards**

A store upgrades its checkout to the new, SCA-ready flow and support tickets start rolling in from returning customers whose saved card "just stopped working." Nothing is wrong with their card. The token WooCommerce saved for them under the old checkout is not something the new checkout can charge. Here is why that happens and a small script that finds every card like this and clears it, so the shopper is asked to re-enter their card once instead of getting a silent decline forever.

[repair_legacy_tokens.py](#)

<https://www.allanninal.dev/woocommerce/legacy-to-new-checkout-cards/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/legacy-to-new-checkout-cards>

62 **Limited-payment coupon miscounts**

A coupon is set to discount only a subscriber's first few renewal payments, say the first three, then go away. Somewhere along the way that stops being true. A subscriber gets the discount on payment five, or it disappears on payment two. The one number WooCommerce Subscriptions uses to track "how many payments has this coupon already discounted" has lost count. Here is why that counter drifts and a small job that recounts it from real paid orders and repairs the rest.

[recount_limited_payment_coupons.py](#)

<https://www.allanninal.dev/woocommerce/limited-payment-coupon-miscounts/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/limited-payment-coupon-miscounts>

63 **Match payouts to orders**

The deposit lands in your bank account. It is one round number for one day. Nothing in WooCommerce tells you which orders are inside it, and the number itself never equals the sum of the order totals from that day anyway. Here is why a payout never lines up with your orders on its own, and a small report that matches every payout line back to a WooCommerce order and ties out to the cent.

[build_payout_report.py](#)

<https://www.allanninal.dev/woocommerce/match-payouts-to-orders/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/match-payouts-to-orders>

64 **The Stripe webhook cannot find the order, so it never updates**

Your Stripe logs are full of one line: Could not find order via intent ID. The payment went through, but WooCommerce never moved the order because the PaymentIntent ID was never saved onto it. The webhook arrives, looks for the order, finds nothing, and gives up. Here is how to recover the lost ID from Stripe and write it back so these orders can be matched, completed, and refunded again.

[backfill_intent_id.py](#)

<https://www.allanninal.dev/woocommerce/missing-intent-id-webhook-cannot-match-order/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/missing-intent-id-webhook-cannot-match-order>

65 **Move sources to payment methods**

A shopper saved their card years ago, and Stripe stored it as an old style Source token that starts with src_. That was fine back then. It is not fine now. Stripe is retiring Sources for saved cards, and a Source was never built to carry a shopper through Strong Customer Authentication (SCA) on a later off-session charge. Here is why that quietly breaks renewals and repeat purchases, and a small script that finds every saved Source and moves it to a PaymentMethod in bulk, safely.

[migrate_sources_to_pm.py](#)

<https://www.allanninal.dev/woocommerce/move-sources-to-payment-methods/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/move-sources-to-payment-methods>

66 **Move WooPayments to Stripe without re-asking buyers for their card**

You are leaving WooPayments for your own direct Stripe account. The dashboard move goes fine, new charges hit the new account, but every saved card on file was issued under WooPayments, and subscriptions quietly start failing to renew. You do not want to email your customers and ask them to type their card back in. Here is why the tokens break and a small script that re-points them once Stripe confirms they are safe to use.

[migrate_woopayments_tokens.py](#)

<https://www.allanninal.dev/woocommerce/move-woopayments-to-stripe/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/move-woopayments-to-stripe>

67 **A new card is not linked to the WooCommerce subscription**

A customer updated their card because the old one was about to expire, and the change looked like it worked. But the next renewal charged the old card anyway and failed. The new card was saved to their Stripe customer, but the subscription still points at the old token. This guide repoints each subscription to the customer's current card so renewals use the right one.

[repoint_sub_card.py](#)

<https://www.allanninal.dev/woocommerce/new-card-not-linked-to-subscription/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/new-card-not-linked-to-subscription>

68 **Next payment date drifts after a late renewal**

A renewal payment failed, got retried a few days late, or ran through a delayed cron job, and now the subscription's next payment date is not where it should be. It is not a huge jump, usually a day or two, but it never corrects itself. Every late renewal after that nudges the date a little further off schedule, until a customer notices they were billed on a different day than they signed up for. Here is why the schedule drifts and a small script that recomputes the correct date and repairs it.

[fix_next_payment_drift.py](#)

<https://www.allanninal.dev/woocommerce/next-payment-date-drifts-after-a-late-renewal/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/next-payment-date-drifts-after-a-late-renewal>

69 Old SEPA sources rejected

A subscription has renewed the same way for a year, then one month Stripe rejects the charge outright. Nothing about the shopper's bank account changed. What changed is Stripe: the saved payment token on that subscription is a legacy Source, and Stripe no longer accepts old Sources for off session SEPA Direct Debit renewals. Here is why it happens and a small script that finds every affected order and migrates it to a supported payment method.

[migrate_sepa_sources.py](#)

<https://www.allanninal.dev/woocommerce/old-sepa-sources-rejected/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/old-sepa-sources-rejected>

70 WooCommerce products show on sale when they are not

The sale ended last week, or someone cleared the sale price in bulk, but the product page still shows the red sale badge and the strikethrough price. Shoppers add it to cart expecting the discount, then feel misled at checkout when the full price rings up. This is a caching problem, not a pricing problem. Here is why the flag gets stuck and a small script that recomputes it from the actual prices on every product.

[recompute_on_sale.py](#)

<https://www.allanninal.dev/woocommerce/on-sale-flag-shows-products-not-on-sale/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/on-sale-flag-shows-products-not-on-sale>

71 Order edits do not adjust stock

A shop manager opens an order, bumps a quantity from one to three, saves it, and moves on. WooCommerce never touches product stock for that change. The order total updates. The line item updates. The number of units sitting in your warehouse stays exactly where it was before the edit. Here is why that happens and a small script that finds every line item where the order and the stock have drifted apart, then brings them back in line.

[reconcile_stock.py](#)

<https://www.allanninal.dev/woocommerce/order-edits-do-not-adjust-stock/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/order-edits-do-not-adjust-stock>

72 WooCommerce orders marked paid with no matching Stripe charge

An order shows as Processing or Completed, so your reports count it as a sale and your team ships it. But when you look in Stripe, there is no charge behind it. The money was never taken. It can come from a manual status change, a plugin that got it wrong, or a tampered checkout, and it quietly inflates revenue and lets goods walk out the door unpaid. Here is why it happens and a small job that checks every paid order against Stripe and flags the ones that do not add up.

[verify_paid.py](#)

<https://www.allanninal.dev/woocommerce/order-marked-paid-no-charge/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/order-marked-paid-no-charge>

73 **WooCommerce order stats wrong after an HPOS migration**

The orders themselves look fine. Customers were charged, order pages show the right totals, and stock moved the way it should. But WooCommerce Analytics tells a different story: revenue is lower than it should be, order counts are off, and a handful of dates look strangely empty. This started right after the store moved to High-Performance Order Storage. Here is why the stats tables fall out of sync and a small script that finds every order they disagree on and rebuilds them safely.

[resync_order_stats.py](#)

<https://www.allanninal.dev/woocommerce/order-stats-wrong-after-hpos-migration/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/order-stats-wrong-after-hpos-migration>

74 **Order tax off by a cent (frontend vs API)**

A buyer looks at the tax line during checkout, and it says one number. A day later, someone pulls the order through the REST API for accounting, and the tax total is a cent or two different. Nobody changed anything. Both numbers were rounded correctly, just at a different point in the math. Here is why that happens and a small script that finds every order where the drift is real and fixes it, or flags it when the gap is too big to guess at.

[reconcile_order_tax.py](#)

<https://www.allanninal.dev/woocommerce/order-tax-off-by-a-cent-frontend-vs-api/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/order-tax-off-by-a-cent-frontend-vs-api>

75 **WooCommerce orders stuck on 3D Secure with Stripe**

A buyer reached the 3D Secure step, then closed the tab or lost the bank redirect. The Stripe PaymentIntent is frozen on `requires_action`, the WooCommerce order sits on Pending, and the stock stays held. Some of these quietly complete a few minutes later and become paid but still Pending. This guide sweeps them all, completes the ones that paid, and frees the ones that did not.

[resolve_3ds.py](#)

<https://www.allanninal.dev/woocommerce/orders-stuck-requires-action-3ds/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/orders-stuck-requires-action-3ds>

76 **Orphaned customers and cards: Stripe customers with no WooCommerce user behind them**

Somewhere in your Stripe dashboard there are customer records with saved cards that no WooCommerce account claims any more. Somewhere in your WooCommerce database there are user accounts pointing at a Stripe customer id that Stripe no longer recognizes. Neither side notices on its own. Here is why the link between a WooCommerce user and its Stripe customer breaks, and a small script that finds every case, reconnects the ones that can be reconnected, and safely cleans up the ones that cannot.

[find_orphaned_customers.py](#)

<https://www.allanninal.dev/woocommerce/orphaned-customers-and-cards/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/orphaned-customers-and-cards>

77 Orphaned WooCommerce postmeta rows from deleted orders

An order gets deleted, but a handful of its rows in wp_postmeta do not go with it. Nothing points to them anymore, but they sit in the table anyway, quietly adding rows to a table that is already one of the biggest in most WooCommerce databases. Here is why the cleanup gets skipped and a small script that finds every orphan before you touch the database with a delete statement.

[find_orphaned_postmeta.py](#)

<https://www.allanninal.dev/woocommerce/orphaned-postmeta-rows/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/orphaned-postmeta-rows>

78 Orphaned product variations: cleaning up variations with no parent product

A product gets deleted, trashed, or switched from a variable product back to a simple one, and somewhere in the database its child variations are still sitting there, each one a real post of its own with nothing left to load it under. Nobody sees them in the catalog, but they still show up in stock reports, background jobs, and old order line items. Here is why WooCommerce leaves them behind and a small script that finds every orphan and cleans it up safely.

[find_orphaned_variations.py](#)

<https://www.allanninal.dev/woocommerce/orphaned-product-variations/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/orphaned-product-variations>

79 Orphaned subscriptions with no customer

Somewhere in your subscriptions list there are rows still marked Active, still renewing, still taking payment from a saved card, that belong to no one. The customer account behind them is gone, or was never really there, and customer_id just reads 0. Stripe does not care and keeps billing the card every cycle. Nobody in your account area can see the order, and no email reaches a real inbox when something goes wrong. Here is why a subscription loses its owner and a small script that finds every orphan, reattaches the ones it safely can, and flags the rest for a human.

[find_orphaned_subscriptions.py](#)

<https://www.allanninal.dev/woocommerce/orphaned-subscriptions-with-no-customer/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/orphaned-subscriptions-with-no-customer>

80 Out of stock but still purchasable

The product page says "Out of stock" in plain red letters, and the buy button still adds it to the cart. A customer checks out, pays, and now you have an order for something you do not have. This is not a rare glitch. It is what happens when the fields WooCommerce uses to decide "can this be bought" quietly drift apart from the field that says "do we have any." Here is why it happens and a small job that finds every product in that state, locks it down, and flags any order that already slipped through.

[fix_purchasable_stock.py](#)

<https://www.allanninal.dev/woocommerce/out-of-stock-but-still-purchasable/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/out-of-stock-but-still-purchasable>

81 **WooCommerce negative stock from an overselling race**

A product had two left. During a busy minute, three people checked out at once, and now its stock reads minus one. Real stock cannot go below zero, but WooCommerce shows it because two orders both passed the stock check before either one saved. That negative number then throws off your reports and your reorder points. Here is why the race happens and a small job that finds the oversold products and resets them in a safe way.

[fix_negative_stock.py](#)

<https://www.allanninal.dev/woocommerce/overselling-race-stock-negative/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/overselling-race-stock-negative>

82 **WooCommerce orders stuck on Pending after a successful Stripe payment**

The card went through. Stripe shows the money. The buyer got charged. But the WooCommerce order still says Pending payment, no email went out, and stock is still held. This is the most common Stripe problem a WooCommerce store will ever file a ticket about. Here is why it happens and a small script that finds every affected order and finishes it in a safe way.

[reconcile_pending.py](#)

<https://www.allanninal.dev/woocommerce/paid-orders-stuck-on-pending/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/paid-orders-stuck-on-pending>

83 **Partial capture total mismatch**

You captured part of an authorized Stripe charge, maybe because only half the order shipped, or stock ran short on one item. Stripe shows the smaller amount you actually took. The WooCommerce order still shows the full original total, as if nothing changed. Now your books do not agree with your payment processor, and nobody told the order. Here is why that gap opens up and a small script that lines the total up with what was really taken.

[sync_partial_capture.py](#)

<https://www.allanninal.dev/woocommerce/partial-capture-total-mismatch/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/partial-capture-total-mismatch>

84 **Partial refund gives back everything**

A shop manager clicks refund for ten dollars on an eighty dollar order. A minute later the customer messages back confused, because Stripe just returned the entire eighty. Nobody touched the amount field twice. Nobody fat fingered a number. The order was captured outside the normal flow, and WooCommerce's refund math never knew that. Here is why it happens and a small script that catches the gap before it drains a payout.

[audit_refunds.py](#)

<https://www.allanninal.dev/woocommerce/partial-refund-gives-back-everything/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/partial-refund-gives-back-everything>

85 **WooCommerce renewals fail because the Stripe payment method was detached**

A subscription that renewed fine for months suddenly starts failing with a plain, unhelpful decline. Nothing changed on the order. Nothing changed in WooCommerce. What changed is on Stripe, the saved card was detached from the customer, so there is nothing left to charge. Here is why that happens and a small script that finds every subscription this hits and flags it so a human can ask for a new card.

[find_detached_payment_methods.py](#)

<https://www.allanninal.dev/woocommerce/payment-method-detached/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/payment-method-detached>

86 Persistent cart data never cleared

Every time a logged in customer touches their cart, WooCommerce saves a copy to a row in wp_usermeta called the persistent cart, so the cart survives a logout or a new device. Nothing ever deletes that row. Years later, a store can have hundreds of thousands of customer accounts each holding a stale cart full of products that were discontinued long ago. Here is why it happens and a small script that finds the stale carts and clears them.

[clear_stale_carts.py](#)

<https://www.allanninal.dev/woocommerce/persistent-cart-data-never-cleared/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/persistent-cart-data-never-cleared>

87 Popularity sort uses stale total sales

Sort a WooCommerce shop by "Popularity" and it should show what actually sells first. Instead it often shows products that stopped selling months ago, while a genuine bestseller sits on page three. The sort is not broken. The number it sorts by, total_sales, is wrong. Here is why that number drifts and a small script that recounts it from real orders.

[recount_total_sales.py](#)

<https://www.allanninal.dev/woocommerce/popularity-sort-uses-stale-total-sales/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/popularity-sort-uses-stale-total-sales>

88 Presentment vs settlement currency

The buyer paid one currency and you settled another. WooCommerce stores a clean order total in the currency the buyer saw at checkout, and never asks what Stripe actually did with the money after that. Stripe can convert the charge into your payout currency at its own rate, on its own schedule, and nothing in WooCommerce writes that conversion down. Here is why the two numbers quietly disagree, and a small script that finds every affected order and makes the exchange explicit.

[record_settlement_currency.py](#)

<https://www.allanninal.dev/woocommerce/presentment-vs-settlement-currency/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/presentment-vs-settlement-currency>

89 Product lookup table out of sync

A shop manager changes a price, saves it, and the storefront still shows the old number. Or a product says in stock everywhere except the low stock report, which insists it sold out days ago. Nothing in the product editor looks wrong. The real data is fine. The problem is a second copy of that data, a lookup table WooCommerce keeps for fast filtering and sorting, and that copy has drifted from the truth. Here is why it happens and a small script that finds the rows that are wrong and rebuilds them.

[rebuild_lookup_rows.py](#)

<https://www.allanninal.dev/woocommerce/product-lookup-table-out-of-sync/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/product-lookup-table-out-of-sync>

90 Product visibility terms mis-assigned: when the catalog settings and the storefront disagree

A product's edit screen says Catalog visibility is Shop and search, it is not marked Featured, and it is in stock. Yet on the storefront it is missing from search, buried out of the catalog, or stuck looking featured when it should not be. The settings are right. The hidden taxonomy terms WooCommerce actually reads at render time are wrong. Here is why the two drift apart and a small script that recomputes the correct terms for every product and repairs the ones that are wrong.

[repair_visibility_terms.py](#)

<https://www.allanninal.dev/woocommerce/product-visibility-terms-mis-assigned/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/product-visibility-terms-mis-assigned>

91 **Products stranded with no category**

The product is published. The price is set. Stock is in. Someone can even buy it if they land on the exact page. But it never shows up on any category page, never appears in a category widget, and never shows in a menu link that filters by category, because it has no category at all. This is easy to miss because nothing looks broken in the product editor. Here is why it happens and a small script that finds every stranded product and gives it a safe fallback category.

[assign_fallback_category.py](#)

<https://www.allanninal.dev/woocommerce/products-stranded-with-no-category/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/products-stranded-with-no-category>

92 **Promote the default source to a real PaymentMethod**

A subscription has been renewing fine for years, then one day it just stops. The card has not expired, the customer has not disputed anything, and nothing in the WooCommerce order looks wrong. The real problem is sitting quietly on the Stripe customer object, the thing everything else charges against is still a legacy Source, and Sources cannot be confirmed off session once Strong Customer Authentication applies. Here is why that happens and a small script that finds every customer stuck like this and promotes a proper PaymentMethod as the default.

[promote_default_source.py](#)

<https://www.allanninal.dev/woocommerce/promote-the-default-source/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/promote-the-default-source>

93 **Proration miscalculated on a second switch**

A customer upgrades a plan, then upgrades again a few days later in the same billing cycle. The first switch order looks right. The second one does not. The credit is too small, too large, or missing, and the order total no longer matches what Stripe actually charged. Here is why the second switch prorates from the wrong number and a small script that finds every switch order where the math went wrong.

[detect_switch_proration.py](#)

<https://www.allanninal.dev/woocommerce/proration-miscalculated-on-a-second-switch/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/proration-miscalculated-on-a-second-switch>

94 **Push a card change to Stripe**

A customer changes their card in WooCommerce, the new card is charged, the order goes through without a hitch. Everyone assumes the job is done. Then the next renewal in Stripe reaches for a card that no longer works, because Stripe was never told the card changed. Here is why the Stripe customer record falls behind and a small script that pushes the new card to Stripe as the default.

[push_card_to_stripe.py](#)

<https://www.allanninal.dev/woocommerce/push-a-card-change-to-stripe/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/push-a-card-change-to-stripe>

95 **WooCommerce product rating count drifts from the real reviews**

A shopper opens a product and sees "4.8 out of 5 (312 reviews)" at the top, then scrolls down and counts a fraction of that many reviews with stars on them. The number at the top is not a live count. It is a cached figure WooCommerce keeps on the product, and every so often that cache falls out of step with what is actually approved. Here is why it drifts and a small script that recomputes it from the real reviews and only touches the products that are actually wrong.

[recount_ratings.py](#)

<https://www.allanninal.dev/woocommerce/ratings-count-drifts-from-real-reviews/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/ratings-count-drifts-from-real-reviews>

96 **Record Stripe fees on WooCommerce orders**

Your WooCommerce reports say you made a certain amount this month. Your bank says less. The gap is the Stripe processing fee, taken on Stripe's side and never written back to the order, so every report you run shows revenue before fees and overstates your profit. Here is why the fee goes missing and a small job that reads it from Stripe and records the fee and net on each order, so your numbers finally reflect what you kept.

[record_fees.py](#)

<https://www.allanninal.dev/woocommerce/record-stripe-fees-on-orders/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/record-stripe-fees-on-orders>

97 **Recreate WooCommerce Subscriptions after a Stripe account move**

The store moved to a new Stripe account, and now renewals are failing with errors like "No such customer" or "No such payment_method." Nothing is wrong with the subscriptions themselves, they are just pointing at customer and card ids that only ever existed on the old account. Here is why that happens and a small script that finds every affected subscription and recreates it on the new account, one chargeable token at a time.

[recreate_subs.py](#)

<https://www.allanninal.dev/woocommerce/recreate-subs-after-account-move/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/recreate-subs-after-account-move>

98 **Recurring coupon dropped on switch**

A customer had a recurring discount on their plan for months. Then they upgraded, downgraded, or moved to a different variation, and the discount quietly stopped applying. The switch order looks fine. The new subscription looks fine. But every renewal after the switch bills the full price, and nobody notices until the customer does. Here is why the switch drops the coupon and a small script that finds it and puts it back.

[reapply_switch_coupon.py](#)

<https://www.allanninal.dev/woocommerce/recurring-coupon-dropped-on-switch/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/recurring-coupon-dropped-on-switch>

99 Refund and dispute double reversal

You already refunded the order. The buyer got their money back, the WooCommerce order says Refunded, and the case looked closed. Weeks later the same buyer disputes the same charge with their bank, and Stripe pulls the money out again, plus a dispute fee. Your balance is now down twice for one sale, and because refunds and disputes live in different reports, almost no one catches it. Here is why it happens and a small script that measures exactly how much you lost.

[refund_dispute_double_reversal.py](#)

<https://www.allanninal.dev/woocommerce/refund-and-dispute-double-reversal/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/refund-and-dispute-double-reversal>

100 Stripe refunds on non-card methods never mark the order Refunded

You refunded an order paid with iDEAL, EPS, giropay, or SEPA, and the money went back on Stripe. But the WooCommerce order still shows its full total and never moved to Refunded. The webhook arrived, it just skipped the order because the handler only recognised plain card payments. This guide finds the orders whose refund was silently dropped and records it, then marks the fully refunded ones Refunded.

[sync_apm_refunds.py](#)

<https://www.allanninal.dev/woocommerce/refund-webhook-skips-non-card-methods/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/refund-webhook-skips-non-card-methods>

101 Remap WooCommerce customers to the right orders after a store move

The migration finished, the new site looks fine, and then the tickets start. "I don't see my past orders." "My subscription shows on someone else's account." A store move rewrites WordPress user IDs, but every order on the old site was saved with the old numeric customer_id. Stripe still knows exactly who paid. Here is why the link breaks and a small script that remaps every order to the correct customer using the trail Stripe left behind.

[remap_customers.py](#)

<https://www.allanninal.dev/woocommerce/remap-customers-after-a-store-move/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/remap-customers-after-a-store-move>

102 Remove test mode Stripe IDs from live WooCommerce orders

Somewhere in your live store, an order is holding a Stripe PaymentIntent id that only exists in test mode. It happened during a migration, a staging push, or a developer checking out with the wrong keys turned on. The order looks fine in WooCommerce, right up until something tries to use that id against your live Stripe account and gets a flat "no such payment_intent" back. Here is why it happens and a small script that finds every one of these orders and clears the bad reference safely.

[remove_test_ids.py](#)

<https://www.allanninal.dev/woocommerce/remove-test-ids-from-live/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/remove-test-ids-from-live>

103 **Renewal actions stall, no renewals made**

Subscriptions still show Active. Customers still expect to be billed. But nothing is happening. No renewal orders are appearing, no charges are going out, and the next payment date on every affected subscription just sits there in the past. The scheduled renewal actions have stalled entirely. Here is why that happens and a small script that finds every subscription stuck this way and triggers the renewal it was owed.

[trigger_stalled_renewals.py](#)

<https://www.allanninal.dev/woocommerce/renewal-actions-stall-no-renewals-made/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/renewal-actions-stall-no-renewals-made>

104 **Renewal charged, no order made**

Stripe shows a renewal that went through. The customer's card was billed, the invoice is marked paid, and the subscription's next billing date has already moved forward. But WooCommerce has no renewal order for that charge anywhere in the order list. No confirmation email went out, nothing was added to their order history, and your books are missing a sale that already happened. Here is why the order never gets made and a small script that finds every orphaned renewal charge and creates the order it should have had.

[create_missing_renewal.py](#)

<https://www.allanninal.dev/woocommerce/renewal-charged-no-order-made/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/renewal-charged-no-order-made>

105 **A subscription renewal is marked paid, but no payment ever happened**

The renewal order says Processing. The subscription's next payment date rolled forward. The customer still has access. Everything in WooCommerce looks correct. Except Stripe never took a charge for it. No PaymentIntent, no invoice, no money. This is quiet, because nothing errors and no one complains, until the payout report comes up short. Here is why it happens and a small script that finds every renewal it happened to and flags it for review.

[flag_fake_paid_renewals.py](#)

<https://www.allanninal.dev/woocommerce/renewal-marked-paid-with-no-payment/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/renewal-marked-paid-with-no-payment>

106 **WooCommerce Subscriptions renewals never run because Action Scheduler stalled**

Subscriptions sit there marked Active. The renewal date came and went. No card was charged, no order was created, and no email went out. Nothing looks broken in wp-admin, but the money stopped moving days ago. This is what happens when the Action Scheduler queue that drives every renewal quietly stalls. Here is why it happens and a small job that finds the backlog and charges the renewals that are actually due.

[run_due_renewals.py](#)

<https://www.allanninal.dev/woocommerce/renewals-never-run/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/renewals-never-run>

107 **Replay the Stripe webhook events WooCommerce missed during downtime**

Your store went down, maybe for maintenance or an outage, for longer than Stripe keeps retrying. When it came back, a batch of orders were out of sync because the events that would have updated them were never delivered. This guide pulls the undelivered events from Stripe and reapplies them, so your orders catch up to where they should be.

[replay_events.py](#)

<https://www.allanninal.dev/woocommerce/replay-missed-stripe-webhook-events/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/replay-missed-stripe-webhook-events>

108 **REST pagination breaks on large sets**

An export job or sync script pages through /wp-json/wc/v3/orders with page and per_page, and it runs fine on a quiet store. On a busy one, the same job quietly comes back short. A handful of orders never show up in any page, and nobody notices until a report does not add up or a paid order never gets synced anywhere. Here is why the pagination misses rows and a small script that walks the same orders with a stable sort instead.

[paginate_orders.py](#)

<https://www.allanninal.dev/woocommerce/rest-pagination-breaks-on-large-sets/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/rest-pagination-breaks-on-large-sets>

109 **Rounding drifts the order total by a cent**

The order total on WooCommerce says 49.99. Stripe says it charged 50.00. Nobody typed a wrong number, no plugin misfired, and the buyer paid exactly what checkout showed them. The two totals just landed a cent apart because they were rounded in two different places. Here is why that happens and a small script that finds every order where the drift is real rounding and every order where it is not.

[detect_rounding_drift.py](#)

<https://www.allanninal.dev/woocommerce/rounding-drifts-the-order-total-by-a-cent/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/rounding-drifts-the-order-total-by-a-cent>

110 **SEPA renewal stays active on fail**

A subscription renewed with SEPA Direct Debit. Stripe said the payment was on its way, WooCommerce marked the renewal Completed, and the subscription stayed Active. Then, more than a week later, the customer's bank returned the debit as failed. Nobody noticed, because that second event never reached the store. The subscription is still active, the customer is still using what they did not pay for, and dunning never started. Here is why the delayed failure gets missed and a small script that catches it and moves the order to On hold so retries can run.

[repair_sepa_renewal.py](#)

<https://www.allanninal.dev/woocommerce/sepa-renewal-stays-active-on-fail/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/sepa-renewal-stays-active-on-fail>

111 **SEPA subscriptions flip to manual renewal after an update**

The mandate is still good. The customer's bank account has not changed. But after a plugin or WordPress update, a batch of SEPA Direct Debit subscriptions quietly switched from automatic to manual renewal. WooCommerce stops charging them on its own, starts emailing invoices instead, and the first sign of trouble is a wave of "why do I have to pay by hand now" messages. Here is why it happens and a small script that turns automatic billing back on for the subscriptions that still have a working mandate.

[restore_sepa_renewal.py](#)

<https://www.allanninal.dev/woocommerce/sepa-subscription-flip-to-manual-renewal/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/sepa-subscription-flip-to-manual-renewal>

112 **The WooCommerce session table that grows without bound**

Every shopper who opens your store, whether they buy or not, gets a row in `wp_woocommerce_sessions`. WooCommerce is supposed to delete those rows once they expire. On plenty of stores that cleanup quietly stops working, and the table just keeps growing, slowing down every page that touches it. Here is why the cleanup breaks and a small script that clears it back down safely, without touching a shopper who is checking out right now.

[clear_stale_sessions.py](#)

<https://www.allanninal.dev/woocommerce/session-table-balloons/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/session-table-balloons>

113 **Slow methods succeed but never match**

SOFORT and Klarna do not confirm at checkout the way a card does. The buyer is sent back to your store with an order still on Pending, and the real confirmation shows up later from a bank or a lender, sometimes hours later. When that follow up message is missed, the `PaymentIntent` quietly moves to succeeded on Stripe while the order never catches up. Here is why the two fall out of sync and a small script that finds the late charge and matches it back to its order.

[match_delayed_payments.py](#)

<https://www.allanninal.dev/woocommerce/slow-methods-succeed-but-never-match/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/slow-methods-succeed-but-never-match>

114 **A staging site paused real, paying subscriptions**

Someone spun up a staging copy of the store to test a plugin update. It looked isolated. It was not. Staging kept the live WooCommerce API keys and the live Stripe secret key, and when its own renewal cron ran and failed for a reason that had nothing to do with the customer, WooCommerce Subscriptions paused the subscription it thought it had just tried to bill, and that subscription was the real one. Billing stopped for paying customers who did nothing wrong. Here is why it happens and a small script that finds the wrongly paused subscriptions and restores the ones Stripe confirms were actually paid.

[restore_wrongly_paused_subs.py](#)

<https://www.allanninal.dev/woocommerce/staging-site-pauses-live-subscriptions/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/staging-site-pauses-live-subscriptions>

115 **Stale Stripe PaymentIntent IDs after a WooCommerce gateway switch**

You moved to a new Stripe account, switched Stripe from test mode to live, or replaced the gateway entirely. The new payments work fine. But the moment someone tries to refund an old order, renew a subscription, or run a sync job against it, Stripe answers with "No such payment_intent". The order is not broken, it is just holding onto an ID from a account that no longer applies. Here is why that happens and a small script that finds every order with a stale ID and clears it safely.

[clear_stale_intent_ids.py](#)

<https://www.allanninal.dev/woocommerce/stale-ids-after-a-gateway-switch/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stale-ids-after-a-gateway-switch>

116 **Stale reserved-stock rows oversell**

A shopper hits checkout and WooCommerce quietly reserves the last unit for them, before a card is even charged. That reservation is supposed to disappear a few minutes later if the sale does not go through. But when the checkout crashes, the tab gets closed, or a background job that clears old holds never runs, the reservation row just sits there. The unit stays "sold" to a buyer who never paid, a second real buyer gets told it is out of stock, or worse, both orders get let through and you oversell. Here is why the reservation goes stale and a small script that finds every expired hold and releases it, without ever touching an order that actually paid.

[release_stale_reservations.py](#)

<https://www.allanninal.dev/woocommerce/stale-reserved-stock-rows-oversell/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stale-reserved-stock-rows-oversell>

117 **Stripe keeps billing a WooCommerce subscription after it was cancelled**

A customer cancels, or you cancel it for them, and WooCommerce shows the subscription as Cancelled. Everyone moves on. Then a month later the same card gets charged again, because Stripe never got the message. WooCommerce closed its side of the deal. Stripe kept running its own billing clock. Here is why that gap opens up and a small script that closes the Stripe side before the next renewal date.

[cancel_stripe_subscription.py](#)

<https://www.allanninal.dev/woocommerce/stripe-bills-after-cancellation/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stripe-bills-after-cancellation>

118 **A Stripe dashboard refund never reached WooCommerce**

Someone refunded a customer straight from the Stripe dashboard. The money went back, but WooCommerce never heard about it. The order still shows Completed with its full total, your reports count money you no longer have, and the numbers stop matching Stripe. This guide records the missing refund in WooCommerce, safely, without ever refunding the customer twice.

[sync_refunds.py](#)

<https://www.allanninal.dev/woocommerce/stripe-dashboard-refund-not-synced/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stripe-dashboard-refund-not-synced>

119 **Stripe fee and net stay stale on an order after a refund**

You partially refund an order, and the money goes back on Stripe. But the Stripe fee and net that WooCommerce saved on the order still show the pre-refund numbers, as if nothing was returned. Any tool that reads those figures for accounting now overstates what you actually kept. This guide recomputes the fee and net from the Stripe balance transaction and writes the correct values back.

[fix_fee_net.py](#)

<https://www.allanninal.dev/woocommerce/stripe-fee-net-stale-after-refund/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stripe-fee-net-stale-after-refund>

120 **Stripe Link becomes manual**

A customer breezes through checkout with Stripe Link, one click, no typing a card number, and the first payment goes through fine. Weeks later the renewal never charges. WooCommerce Subscriptions has quietly switched the subscription to manual renewal, so it emails an invoice and waits for the customer to pay it by hand, and most never do. Here is why Link checkouts can skip the step that saves a reusable payment method, and a small script that finds the subscriptions this happened to and switches the ones it safely can back to automatic.

[restore_automatic_renewal.py](#)

<https://www.allanninal.dev/woocommerce/stripe-link-becomes-manual/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stripe-link-becomes-manual>

121 **The Stripe webhook is never delivered, so no order updates**

Orders come in, payments succeed on Stripe, and nothing moves in WooCommerce. When every order behaves this way, the usual cause is the webhook itself. The endpoint is wrong, it is disabled, or it does not listen for the events WooCommerce needs. This guide checks all three with a short script and shows you exactly where the wiring is broken.

[check_webhook.py](#)

<https://www.allanninal.dev/woocommerce/stripe-webhook-not-delivered-configuration/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stripe-webhook-not-delivered-configuration>

122 **Stripe rejects every webhook because the signing secret does not match**

Your endpoint is set up, the events are selected, and Stripe still shows every delivery failing with No signatures found matching the expected signature. Orders never update. The cause is a mismatch between the signing secret saved in the WooCommerce Stripe plugin and the one on the Stripe endpoint. This guide finds that mismatch and tells you exactly what to fix.

[check_webhook_secret.py](#)

<https://www.allanninal.dev/woocommerce/stripe-webhook-signing-secret-mismatch/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stripe-webhook-signing-secret-mismatch>

123 **Stuck in pending-cancel**

A customer cancelled weeks ago. The subscription still shows status pending-cancel, it never moved to cancelled, and it is still sitting in every report as if it might come back. Nobody is being charged, but nothing looks finished either. pending-cancel is supposed to be a short wait for the current paid period to end, not a permanent state. Here is why it gets stuck and a small script that finds every subscription like this and finishes the cancellation safely.

[cancel_stuck_pending_cancel.py](#)

<https://www.allanninal.dev/woocommerce/stuck-in-pending-cancel/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/stuck-in-pending-cancel>

124 **A WooCommerce subscription has no saved card, so every renewal fails**

The first payment went through and the customer is happily subscribed, but every renewal fails with "unable to process your payment at this time." The subscription has no saved Stripe customer or card to charge. It usually starts with a declined first attempt and a successful retry that never wrote the token back. This guide recovers the card from the paid order and puts it back on the subscription.

[backfill_sub_token.py](#)

<https://www.allanninal.dev/woocommerce/subscription-missing-saved-card-token/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/subscription-missing-saved-card-token>

125 **A WooCommerce subscription stays On-Hold after a paid renewal**

The renewal charged fine. Stripe took the money, the renewal order shows Completed, and the order note even says the subscription was set back to active. But the subscription itself is still On-Hold, so a paying customer looks suspended and may lose access to what they paid for. This guide finds those subscriptions and reactivates the ones that were genuinely paid.

[reactivate_paid_subs.py](#)

<https://www.allanninal.dev/woocommerce/subscription-on-hold-after-successful-renewal/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/subscription-on-hold-after-successful-renewal>

126 **Subscription price drifts from the product**

A subscription is supposed to keep the price it was created with, even after the product's price changes later. That part is by design. The real problem shows up when the subscription's own stored total quietly disagrees with what Stripe actually charged on its last renewal, usually left behind by a manual edit, an import, or a currency or tax change. Here is why the two numbers drift apart and a small script that finds every subscription where they no longer agree.

[subscription_price_drift.py](#)

<https://www.allanninal.dev/woocommerce/subscription-price-drifts-from-the-product/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/subscription-price-drifts-from-the-product>

127 **Subscription will not end**

A customer signed up for a plan that was supposed to run for a fixed number of payments, and then stop. Six months after the end date passed, it is still active, still charging the card, and nobody set it to keep going. This is what happens when the one scheduled task that should have stopped it never fires. Here is why it happens and a small script that finds every overdue subscription and expires it in a safe way.

[expire_overdue_subscriptions.py](#)

<https://www.allanninal.dev/woocommerce/subscription-will-not-end/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/subscription-will-not-end>

128 **WooCommerce subscriptions that reverted to manual renewal**

Your subscriptions used to charge themselves. After an update or a gateway change, a batch of them quietly switched to manual renewal. Now WooCommerce emails an invoice and waits for the customer to pay by hand, and most of them do not. The card is still on file and perfectly good, but nothing charges it. Here is why active subscriptions flip to manual and a small job that turns automatic renewal back on for the ones that still have a token.

[restore_auto_renewal.py](#)

<https://www.allanninal.dev/woocommerce/subscriptions-revert-to-manual-renewal/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/subscriptions-revert-to-manual-renewal>

129 Sync products to Stripe

A subscription plan sells fine in WooCommerce. Renewals process. Then someone opens Stripe to check billing and there is no Product, no Price, nothing to point to. Products are missing on the Stripe side, so anything that depends on a real Stripe Price, from Stripe-hosted invoices to reporting to a second integration, has nothing to read. Here is why it happens and a small script that creates the matching Stripe products.

[sync_products_to_stripe.py](#)

<https://www.allanninal.dev/woocommerce/sync-products-to-stripe/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/sync-products-to-stripe>

130 Timezone-corrupted next payment dates

A subscription renews hours before it should, or twice on the same day, or looks overdue when the customer was charged just fine last week. The subscription itself is not broken. The next payment date saved against it is. Somewhere along the way, a local time value was written into a field that WooCommerce Subscriptions and Action Scheduler both expect to hold UTC, and now the schedule is running on a corrupted clock. Here is why that happens and a small script that finds every affected subscription and repairs the ones it can safely fix.

[fix_next_payment_timezone.py](#)

<https://www.allanninal.dev/woocommerce/timezone-corrupted-next-payment-dates/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/timezone-corrupted-next-payment-dates>

131 WooCommerce trashed orders still counted in Analytics stats

You trash a batch of test orders, or delete the junk left over from a spam wave, and everything looks clean in the order list. Then the revenue chart in Analytics still shows the same total it did before. Nothing in the order list explains it, because the order list is not where that number comes from. Here is why trashed orders keep counting and a small script that excludes the ones that should not, without ever hiding a real sale.

[exclude_trashed_from_stats.py](#)

<https://www.allanninal.dev/woocommerce/trashed-orders-still-counted-in-stats/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/trashed-orders-still-counted-in-stats>

132 Trial-end action false-positive failure

Action Scheduler shows the trial end action as failed, red text and all. You brace for an angry customer with a broken subscription. But when you look, the subscription is active, the first renewal order exists, and Stripe shows the charge succeeded. Nothing is actually wrong. Here is why that failed entry shows up anyway, and a small script that checks the real state and clears the false alarm without touching anything that might really be broken.

[clear_trial_end_false_positive.py](#)

<https://www.allanninal.dev/woocommerce/trial-end-action-false-positive-failure/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/trial-end-action-false-positive-failure>

133 Variations stuck On Backorder at zero

A shopper opens a product page and one variation says "On backorder" even though you turned backorders off weeks ago and the stock has been at zero the whole time. Nothing in the admin looks wrong at a glance, the quantity field really does say zero, yet the storefront keeps offering a status that should not exist anymore. Here is why the stored stock_status drifts away from the real numbers, and a small script that puts every variation back in sync.

[fix_variation_stock_status.py](#)

<https://www.allanninal.dev/woocommerce/variations-stuck-on-backorder-at-zero/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/variations-stuck-on-backorder-at-zero>

134 WooCommerce orders dropped by a Stripe webhook API version mismatch

Everything used to work. Then one day orders that Stripe shows as paid stop getting a transaction id, and some of them never leave Pending. Nothing in your code changed. What changed is the shape of the object Stripe sends back, because your webhook handler was written for an old API version that had a charges array on every PaymentIntent, and newer versions do not send that array anymore. Here is why that breaks silently and a small script that finds and repairs every order it dropped.

[resolve_charge_id.py](#)

<https://www.allanninal.dev/woocommerce/webhook-api-version-mismatch/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/webhook-api-version-mismatch>

135 WP-Cron disabled, emails never send

The order came through, the payment is fine, and the customer swears they never got a confirmation email. Support checks the SMTP plugin, the spam folder, everything looks normal. The real cause is quieter than that: WP-Cron is disabled or starved, so the queue that sends every order email, and runs most of WooCommerce's background jobs, has simply stopped moving. Here is why that happens and a small script that detects the stuck queue before customers notice.

[cron_watchdog.py](#)

<https://www.allanninal.dev/woocommerce/wp-cron-disabled-emails-never-send/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/wp-cron-disabled-emails-never-send>

136 Wrong coupon type on renewals

A customer signed up with a "first order only" coupon, and it should have stopped there. Instead every renewal since keeps taking the same discount off the top, month after month. WooCommerce Subscriptions is built to tell recurring coupons from one time coupons, but when a coupon of the wrong type ends up on a renewal anyway, nothing stops it from repeating forever. Here is why it slips through and a small script that finds every renewal carrying the wrong coupon type and strips it out.

[strip_bad_renewal_coupons.py](#)

<https://www.allanninal.dev/woocommerce/wrong-coupon-type-on-renewals/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/wrong-coupon-type-on-renewals>

137 **Zero cost renewal orphaned by block checkout**

A subscriber's renewal came out to \$0.00, a coupon covered it, a switch left a credit, a trial converted with a balance still applied, and nothing needed to be charged. But the renewal order never finished. It sits on Pending or On hold, no receipt went out, and the subscription's next payment date never moved. This is a quiet bug in how the new block checkout handles a free renewal, and a small script that finds every order it left behind and completes it in a safe way.

[complete_zero_cost_renewal.py](#)

<https://www.allanninal.dev/woocommerce/zero-cost-renewal-orphaned-by-block-checkout/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/zero-cost-renewal-orphaned-by-block-checkout>

138 **Zero decimal currency charged 100x**

A store selling in yen or won looks fine in the WooCommerce admin, the order total reads PY5,000, but Stripe shows a charge of PY500,000. Nobody typed an extra two zeros. The checkout code did that, by treating every currency as if it has cents. Here is why zero decimal currencies get multiplied by 100 and a small script that finds every order this happened to and refunds the exact overcharge.

[fix_zero_decimal_overcharge.py](#)

<https://www.allanninal.dev/woocommerce/zero-decimal-currency-charged-100x/>

<https://github.com/allanninal/woocommerce-fixes/tree/main/zero-decimal-currency-charged-100x>
